Time-Locked Attestations

Commit-Reveal as a Runtime Primitive

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-22

Contact: hello@ligate.io

Abstract

A standard attestation publishes its payload at submission time. For some workloads this is wrong: a sealed-bid auction wants commitments now and reveals after close; an embargoed announcement attests now and goes public at announcement time; a regulatory filing must land at deadline but become public only after a retention period. Off-chain commit-reveal protocols handle these cases at the cost of trusting an intermediary to hold the sealed payload until reveal time.

This paper specifies **time-locked attestations** as a runtime primitive. A schema field `reveal_at` declares the block height after which the payload becomes valid for reveal. The commit phase submits a binding commitment $h = H(\text{payload} \| \text{nonce})$ as the attestation. The reveal phase submits $(\text{payload}, \text{nonce})$, which the runtime checks against h and the time-bound. Validators enforce the reveal-window; never-revealed commitments age out via TTL with explicit cleanup semantics.

The mechanism is positioned as a **v1.5 protocol feature**. v1 of Ligate Chain ships single-phase attestations only. v0.2 of this paper begins authoring when at least one design partner per use-case category has submitted a concrete use case.

Three contributions. First, we specify the protocol-level mechanism (§4): `MsgCommit`, `MsgReveal`, `MsgCleanup`, the commitment lifecycle state machine, the optional deposit-on-commit, and the front-running defense via batched-reveal sequencing. Second, we analyze cryptographic soundness (§5): the commit-reveal scheme is binding under collision-resistance of the chosen hash (SHA-256 default) and hiding under sufficient nonce entropy (128-bit minimum); the time-lock semantics are enforced by validators reading chain height, no exotic primitives required. Third, we analyze six failure modes (§8) with bounded damage arguments: never-reveal, late-reveal, front-running between commit and reveal, hash collisions, nonce reuse, and reveal-DoS.

The mechanism is positioned as a **v1.5 protocol feature**, post-devnet but pre-mainnet. v1 ships with single-step attestations only; time-locked attestations land when at least one design-partner use case per category (auction, embargo, regulatory time-lock) is validated. §6 documents the gate. This paper exists so the design space is captured before then, not as a roadmap commitment to ship.

---

1. Introduction

1.1 The Late-Disclosure Thesis

[**v0.2:** Why some attestations should not be readable at submission time. Three use-case families: sealed-bid auctions (commit bid now, reveal after close), embargoed announcements (commit story now, publish at agreed time), regulatory time-locks (file now, public after retention period). Each has a different failure mode and time horizon, but the underlying primitive is the same.]

1.2 The Off-Chain Commit-Reveal Trust Problem

[**v0.2:** Standard off-chain commit-reveal: bidder commits hash to a chain, reveals to an auctioneer at close. Auctioneer is trusted to (a) actually run the auction, (b) not reveal payloads early, (c) tally honestly. Each trust assumption is a real failure mode in production auctions. On-chain commit-reveal eliminates the auctioneer.]

1.3 Why Now (or Why v1.5 Not v1)

[**v0.2:** v1 ships single-phase attestations because the four flagship products (Themisra, Mneme, Iris, Kleidon) do not need commit-reveal at launch. Time-locked attestations matter when use cases like auctions and regulatory filings ramp up. The v1.5 timing reflects this; the primitive is small enough to add post-launch without disruption.]

1.4 The Central Question

> [**v0.2:** What is the minimum on-chain commit-reveal primitive that handles the three use-case families (auction, embargo, regulatory), with adequate cryptographic security and explicit never-reveal cleanup, without re-introducing the trust assumptions of off-chain protocols?]

1.5 Approach in Brief

[**v0.2:** Three-sentence preview. Schema includes a `reveal_at` field. `MsgCommit` publishes $h = H(\text{payload} \| \text{nonce})$; `MsgReveal` publishes $(\text{payload}, \text{nonce})$ and the runtime validates the commitment match and the time-bound. TTL-based cleanup handles never-reveal commitments; deposit-on-commit handles economic incentive to reveal.]

1.6 Contributions

[**v0.2:** Mechanism specification, cryptographic security argument, failure-mode analysis, formal comparison with off-chain commit-reveal and on-chain auction protocols (Vickrey, EIP-7251 family).]

1.6.1 Status of Claims [**v0.2:** Same panel as PoUA v0.7 §1.6.1. Cryptographic security claims are "proven" (under standard hash-function assumptions); never-reveal incentive claims are "bounded-under-stated-assumptions"; use-case fitness is "empirical-or-heuristic."]

1.7 Scope and Non-Goals

[**v0.2:** In scope: commit-reveal as a runtime primitive, three use-case families, never-reveal cleanup. Out of scope: zero-knowledge variants (deferred), Vickrey-specific auction protocols (build on top of this primitive), cross-chain time-locks (separate paper), generic privacy-preserving attestation (different paper).]

1.8 Document Structure

[**v0.2:** TOC walkthrough.]

---

2. Background and Related Work

2.1 Cryptographic Commitments

[**v0.2:** Hash-based commitments: $H(\text{payload} \| \text{nonce})$. Pedersen commitments. Lamport one-way commitments. Tradeoff: hash-based is simple and chain-friendly; Pedersen is additively homomorphic; Lamport is post-quantum-friendly but bandwidth-heavy.]

2.2 On-Chain Auctions

[**v0.2:** Vickrey (second-price sealed-bid), Vickrey-Clarke-Groves, English ascending. Each has different commit-reveal requirements. Existing implementations on Ethereum, with off-chain auctioneer or smart-contract auctioneer.]

2.3 Off-Chain Commit-Reveal Protocols

[**v0.2:** Verifiable Random Function (VRF) commit-reveal, RANDAO, Drand. Each solves a related but distinct problem; informs the cryptographic design.]

2.4 Time-Locked Encryption

[**v0.2:** Drand-based timelock encryption (Drand `quicknet`), VDF-based time-locks (Wesolowski VDF), proof-of-elapsed-time (Intel SGX). All complement on-chain commit-reveal with different threat models.]

2.5 Embargoed-Document Storage Networks

[**v0.2:** Filecoin retrieval markets, Arweave with retrieval delays, IPFS-with-encryption. Different from on-chain commit-reveal but adjacent in problem space.]

---

3. System Model

This section formalizes the commit-reveal model: commitments, reveals, the commitment lifecycle, and the cryptographic primitives the chain leans on. Standard PoUA assumptions (BFT consensus, threshold-signed attestor sets, deterministic block heights) carry over; this section names only what's new for time-locked semantics.

3.1 Commitment

A **commitment** is a tuple

$$c = (h, \text{reveal\_at}, \text{ttl}, \sigma, \mathcal{A}_\sigma, d)$$

where:

- $h \in \{0,1\}^{256}$ is the **commitment hash**, $h = H(\text{payload} \| \text{nonce})$ under the schema's declared hash function H (default SHA-256).
- reveal_at $\in \mathbb{N}$ is the earliest block height at which the reveal becomes valid. Must be strictly greater than the commitment's inclusion height.
- ttl $\in [\text{ttl}_{\min}, \text{ttl}_{\max}]$ is the **time-to-live**: the number of blocks after `reveal_at` during which a valid reveal can land. Past `reveal_at + ttl`, the commitment expires.
- σ is the schema-id; the commitment is bound to the schema's payload type and validation rules.
- $\mathcal{A}_\sigma$ is the schema's attestor set, identifying the threshold-signature group authorized to submit this commitment.
- $d \in \mathbb{R}_{\geq 0}$ is the optional deposit held in escrow until reveal or cleanup (§4.4).

The commitment is published on-chain at commit-time in a `MsgCommit` transaction (§4.1). Once included, it persists in chain state under the schema's commitment table, indexed by canonical `commitment_id` (a hash of the commitment fields plus the inclusion height for uniqueness).

Why time-bound is part of the commitment. A schema can support multiple in-flight commitments with different reveal windows; the commitment binds its own time-lock at commit-time. This is the runtime-primitive design: time-bounds are state, not application logic, so the chain enforces them uniformly.

3.2 Reveal

A **reveal** is the pair (payload, nonce) submitted in a `MsgReveal` transaction (§4.2), at any block height $h \in [\text{reveal\_at}, \text{reveal\_at} + \text{ttl})$. The runtime checks \$H(payload | nonce) = \$ commitment's h. If the check passes, the commitment transitions COMMITTED → REVEALED, the payload becomes the canonical attestation payload (readable by anyone), and any escrowed deposit returns to the committer.

Reveal must come from the same signer as the commit. The reveal carries the same threshold-signature from $\mathcal{A}_\sigma$ as the commit. This prevents a witness who learns the payload off-chain from submitting the reveal first to claim the credit; only the attestor set that committed can reveal.

The reveal is the attestation. Until reveal, only the commitment exists in chain state. The chain does not (yet) know the payload. After reveal, the payload becomes the canonical attestation, indexed under the schema and visible to all readers. From an attestation-graph perspective (cross-schema-composition v0.2 §3.3), the reveal is the moment the schema's downstream consumers see the attestation as valid input.

3.3 Validity State Machine

A commitment's state evolves through a four-state machine:

```
COMMITTED ----> REVEALED              (reveal landed in window)
    |
    +--> EXPIRED ----> CLEANED-UP     (no reveal by reveal_at + ttl)
```

State transitions:

- **COMMITTED**: initial state on inclusion of a successful `MsgCommit`.
- **COMMITTED → REVEALED**: fired by a successful `MsgReveal` in the reveal window. Terminal. Payload becomes canonical; deposit returns.
- **COMMITTED → EXPIRED**: deterministic transition at height $h \geq \text{reveal\_at} + \text{ttl}$. The chain does not require an explicit transaction; expiry is computed from chain state at read time. No payload is published; the commitment "ages out."
- **EXPIRED → CLEANED-UP**: fired by a `MsgCleanup` transaction (§4.3). Removes the commitment from active state, distributing the deposit per the schema's `deposit_destination`. Terminal. A tombstone is retained for query consistency.

Cleanup is permissionless. Anyone can submit `MsgCleanup` for an EXPIRED commitment; the cleanup-runner earns a small protocol fee (§4.3). This creates a permissionless cleanup market: independent operators monitor the chain for expired commitments and submit cleanup transactions. Validators have priority but no monopoly.

Read-time semantics. A read of commitment c at height h returns:

- `state = COMMITTED` if $h <$ reveal_at + ttl and no reveal landed
- `state = REVEALED` plus the payload if reveal landed
- `state = EXPIRED` if $h \geq$ reveal_at + ttl and no reveal landed
- `state = CLEANED-UP` plus tombstone metadata if cleanup ran

The read-time computation is $O(1)$: look up the commitment record, compare current height against `reveal_at + ttl`. No graph traversal.

3.4 Hash Function and Nonce Length

The commitment scheme requires a **collision-resistant** and **pre-image-resistant** hash function. The protocol defaults to SHA-256; schemas may declare alternatives at registration:

- **SHA-256** (default): 256-bit output, post-quantum collision resistance estimated at 128 bits. Universally supported.
- **BLAKE3**: faster on modern hardware, same 256-bit output, same security level. Recommended for high-throughput auction schemas.
- **Poseidon**: ZK-friendly hash. Slower verification but enables zero-knowledge reveal variants (§9.1).

Nonce length minimum: 128 bits. A nonce shorter than 128 bits leaks information about low-entropy payloads (e.g., binary-decision auctions where the payload is one of two values; a 64-bit nonce can be brute-forced offline against the commitment hash). 128 bits is the protocol-level floor; schemas can require longer (256-bit nonces for sensitive embargo schemas, etc.).

Hash function migration. A schema's hash function is set at registration and immutable. To switch (e.g., when SHA-256 falls to a future quantum attack), the schema must register a new schema-id with the new hash; existing commitments under the old schema continue under the old hash. This is the same migration pattern as native-delegation v0.2 §10.4 (PQ signature scheme upgrade): backward-compatible through schema versioning, never retroactive.

Why not require deterministic randomness from the chain? A schema author might want the chain to supply the nonce (e.g., from the block-hash beacon) to remove the committer's freedom in choosing nonces. The chain *could* offer this, but doing so risks committer-validator collusion (a validator who knows the chain-supplied nonce ahead of time can pre-compute possible commitments). v0 keeps the nonce committer-supplied with the 128-bit minimum; future work could explore chain-supplied-nonce variants under explicit threat models.

---

4. Mechanism

This section specifies the three transaction types (`MsgCommit`, `MsgReveal`, `MsgCleanup`), the commitment lifecycle state machine, the optional deposit-on-commit mechanism, and the front-running defense via batched-reveal sequencing.

4.1 Commit Transaction

The **commit transaction** publishes a binding commitment to the chain without revealing the underlying payload.

Schema.

```
MsgCommit = {
  schema_id:         SchemaId,        // bound schema
  signer:            AttestorSetSig,  // threshold sig
  commitment_hash:   Bytes32,         // h = H(payload || nonce)
  reveal_at:         BlockHeight,     // earliest reveal block
  ttl_blocks:        int,             // grace window
  deposit:           TokenAmount,     // optional escrow
  metadata_uri_hash: Bytes32,         // optional off-chain ptr
}
```

Validation at admission.

1. `schema_id` exists in Σ and the signer matches its attestor set $\mathcal{A}_\sigma$.
2. `commitment_hash` is exactly 32 bytes.
3. `reveal_at` $>$ current chain height H (cannot commit-and-reveal in the same block).
4. `ttl_blocks` $\in [\text{ttl}_{\min}, \text{ttl}_{\max}]$ where the bounds are governance-set protocol parameters (defaults: `ttl_min` $= 6$ blocks ≈ 72 seconds, `ttl_max` $= 100{,}800$ blocks ≈ 14 days).
5. `deposit` $\geq$ the schema's `deposit_floor` (registration-time parameter; default 0).
6. Standard PoUA fee payment.

State transition. On success: a new entry is appended to the chain's commitment table with state COMMITTED. Per-block tally counts this for proposer reputation (PoUA §4.3) at the schema's standard fee weight.

Per-schema deposit floor. A schema may declare a non-zero `deposit_floor` at registration. Auction schemas typically set this to a meaningful fraction of the bid value; embargo schemas typically set it to zero. The deposit is held in escrow until reveal or cleanup (§4.4).

4.2 Reveal Transaction

The **reveal transaction** publishes the original payload and proves it matches a prior commitment.

Schema.

```
MsgReveal = {
  commitment_id: CommitmentId,     // canonical id of the prior MsgCommit
  payload: Bytes,                  // original payload
  nonce: Bytes,                    // nonce used in H(payload || nonce)
  signer: AttestorSetSig           // same threshold signature as the commit
}
```

Validation at admission.

1. `commitment_id` exists in the chain's commitment table.
2. The commitment's state is COMMITTED.
3. Current height $H \in [\text{reveal\_at}, \text{reveal\_at} + \text{ttl\_blocks})$.
4. \$H(payload | nonce) = \$ commitment's `commitment_hash` where H is the schema-declared hash function (default SHA-256).
5. The signer is the same attestor-set signature as the commit (prevents reveal by an unauthorized party).

6. Standard PoUA fee payment.

State transition. On success: commitment state transitions COMMITTED → REVEALED. The payload becomes the canonical attestation payload (visible to all readers, indexed by schema). Any deposit held in escrow returns to the committer's address.

Reveal-side reputation. The schema's per-attestation reputation is credited at reveal time, not commit time. This is a deliberate design choice: the commitment alone is not a complete attestation (no payload); only the reveal closes the loop. PoUA §4.3 reputation accrual fires for the reveal signer.

4.3 Cleanup of Expired Commitments

The **cleanup transaction** removes a never-revealed commitment from active chain state.

Schema.

```
MsgCleanup = {
  commitment_id: CommitmentId,     // canonical id
  cleanup_runner: Address          // recipient of cleanup-runner reward
}
```

Validation at admission.

1. `commitment_id` exists.
2. Commitment state is COMMITTED (not yet REVEALED or EXPIRED).
3. Current height $H \geq$ reveal_at + ttl_blocks.

State transition. On success:

1. Commitment state transitions COMMITTED → EXPIRED → CLEANED-UP (atomic in the same block).
2. The commitment record is removed from the active commitment table (state pruning); a tombstone with `(commitment_id, EXPIRED)` is retained for query consistency.
3. Any deposit is distributed per the schema's `deposit_destination`:
   - `COMMITTER`: returned to the original committer (default for low-stakes embargo schemas)
   - `TREASURY`: routed to the chain treasury (default for auction schemas)
   - `BURN`: sent to the pure-burn address per PoUA §5.5.3 (matches the τ_{burn} destination)
   - `CLEANUP_RUNNER`: routed to `cleanup_runner` field (incentivizes anyone to call cleanup on expired commits)
   - Combinations: schemas can declare a split (e.g., 50% to runner + 50% to treasury) to fund cleanup-runner economics

Cleanup-runner economics. A small protocol fee is rebated to `cleanup_runner` regardless of `deposit_destination`. This creates a permissionless cleanup market: anyone monitoring the chain for expired commitments can submit `MsgCleanup` and earn the fee. Validators have priority but no monopoly.

4.4 Deposit-on-Commit Mechanism

The **deposit** field on `MsgCommit` is optional but governed by the schema's `deposit_floor`.

Why deposits matter. Without a deposit, an adversary can spam commitments with no economic cost. Each commitment consumes chain state until cleanup; spam attacks are bounded only by per-tx fees. With a deposit, spam attacks pay the deposit upfront and lose it on never-reveal.

Per-schema policy.

Use case	`deposit_floor`	`deposit_destination`
Sealed-bid auction	high (e.g., $\geq$ minimum bid value)	`BURN` or `CLEANUP_RUNNER` (deters spam, never returns to committer)
Embargoed announcement	zero or symbolic	`COMMITTER` (return on reveal; symbolic on cleanup)
Regulatory time-lock	moderate	`TREASURY` (deters frivolous filings)
Insurance claim	moderate	`COMMITTER` on reveal; `TREASURY` on expiry

Deposit-floor calibration. The `deposit_floor` should make the spam-cost-per-commitment exceed the cost of one `MsgCleanup` execution. At v0 chain parameters (estimated ~0.001 LGT per cleanup), a deposit floor of ~0.01 LGT (10x cleanup cost) is sufficient. Schemas with high-value commitments (e.g., auctions) set the floor much higher to bound the lost-deposit risk for honest committers.

Why deposits, not just fees. The base commit fee is paid even on successful reveal; it does not deter never-reveal because reveal itself doesn't refund the fee. The deposit is the only mechanism that creates economic incentive to reveal: lose the deposit if you don't reveal. Deposits and fees are complementary: fees fund proposer reward / burn, deposits fund cleanup runner / spam deterrence.

4.5 Front-Running Defense

A naive reveal exposes the payload in the mempool before block inclusion. An MEV-aware adversary observing the reveal in the mempool could submit a competing commitment using the now-public payload + a fresh nonce, then race the original reveal to settlement. This is the **commit-reveal front-running attack**.

Defense: batched-reveal sequencing.

The proposer's block-construction rule mandates that **all reveals are sequenced before any new commits in the same block**. Specifically, a block's transaction order is:

```
1. All MsgReveal transactions (sorted by commitment_id; canonical order)
2. All MsgCleanup transactions (sorted by commitment_id)
3. All other transactions (MsgCommit, attestations, transfers, etc.)
```

This ordering is enforced by the runtime: a block whose `MsgCommit` precedes any `MsgReveal` is invalid. Proposers proposing a non-conforming block are slashed per PoUA §4.5.

Why this defends. A reveal in the mempool can be observed, but the adversary cannot submit a competing commitment that lands in the same block before the reveal. The reveal is sequenced first;

the original commitment is finalized; any competing commit lands in a later block. By the time the adversary's competing commit is included, the original payload is already canonical and the adversary's commit is at best a duplicate (rejected by the schema's `commitment_hash` uniqueness check, §4.1 admission).

Limitations.

- **Cross-block races.** The defense protects within a single block. If the reveal takes more than one block to settle (e.g., during network congestion), the adversary has multiple opportunities to compete. Mitigated by the schema declaring a tight `ttl_blocks` (forces resolution quickly) and by validators prioritizing reveals in their mempool.
- **Validator collusion.** A validator who colludes with an adversary can submit the competing commit before the reveal. PoUA §A.1 / §A.2 detectors flag schema-mix anomalies; a validator persistently delaying reveals would deviate from the chain-wide null distribution and trigger slashing. Practical defense.
- **Encrypted-mempool variants.** A future upgrade can use encrypted mempools (à la SUAVE, Shutter Network) to prevent reveal observation entirely. Out of scope for v0; tracked as future work.

Canonical ordering rule. Within the reveals-first batch, reveals are sorted ascending by reveal-at block, then by commitment-id. This is deterministic: every honest validator constructs the same block order given the same mempool, and the rule is verifiable post-hoc by any light client.

---

5. Cryptographic Security

This section formalizes the security properties of the commit-reveal scheme. We show that under standard cryptographic assumptions on the hash function H and nonce entropy, the scheme is binding (committer cannot change the payload after commit) and hiding (readers cannot learn the payload before reveal). The time-lock semantics are enforced by chain consensus, not by exotic cryptography (no VDFs, no time-lock encryption required).

5.1 Binding

Claim. Under collision-resistance of H, a committer cannot change the payload after commit. Given a published commitment h, no adversary can produce two different reveals $(\text{payload}_1, \text{nonce}_1)$ and $(\text{payload}_2, \text{nonce}_2)$ that both pass the reveal check $H(\cdot) = h$ with $\text{payload}_1 \neq \text{payload}_2$.

Argument. Producing two reveals that hash to the same commitment is a second-preimage attack on H. SHA-256 second-preimage resistance is $\sim 2^{256}$ (stronger than the 2^{128} collision bound). To construct a collision-by-design would require approximately 2^{128} hash computations under the birthday bound; second-preimage against a specific known hash h is harder. Both are infeasible with current or projected hardware.

Limit of the claim. Binding is conditional on H being second-preimage-resistant. If a future cryptanalytic break reduces SHA-256 below 2^{80} work, schemas using SHA-256 must migrate via §3.4. Existing commitments under the old hash inherit the weakened binding; schemas handling high-value commitments should monitor cryptanalytic developments and rotate proactively.

5.2 Hiding

Claim. Under pre-image-resistance of H and sufficient nonce entropy, a reader of the commitment cannot learn the payload before reveal. Given h, no efficient adversary (without side-channel access to the payload) can recover payload with probability significantly better than $|\text{payload-space}| \cdot 2^{-|\text{nonce}|}$.

Argument. The adversary's best attack is exhaustive enumeration: for each candidate payload p and each candidate nonce n, compute $H(p\|n)$ and check against h. Total work: $|\text{payload-space}| \cdot 2^{|\text{nonce}|}$ hash computations.

For a binary-decision auction (payload-space $= \{\text{YES}, \text{NO}\}$) with a 128-bit nonce, the effective security is $2 \cdot 2^{128} \approx 2^{129}$ work. Infeasible.

For an embargoed news article (payload-space essentially unbounded text, but with low entropy in the headline / first paragraph), the work is bounded below by $2^{|\text{nonce}|}$. The 128-bit floor delivers 128-bit hiding regardless of payload structure.

Caveat: side channels. Hiding is broken if the adversary has any off-chain knowledge of the payload (overheard a bidder, accessed the committer's logs). The cryptographic argument bounds attacks operating purely on the commitment hash; it does not protect against operational leakage. Application-layer guidance: committers should be operationally careful with the payload-nonce pair until reveal.

5.3 Nonce Entropy Bound

Claim. The minimum nonce length of 128 bits is the protocol-level floor. Schemas may require more.

Argument. Per §5.2, hiding security scales as $|\text{payload-space}| \cdot 2^{|\text{nonce}|}$. For a low-entropy payload space (binary, ternary, or any small discrete set), work is dominated by the nonce length. 128 bits gives 128-bit security against brute force; 64 bits would be brute-forceable on a small GPU cluster in days (2^{64} SHA-256 hashes is $\sim 10^{15}$ ops).

Schema-declared nonce minimums. A schema can declare a longer minimum at registration (e.g., 256 bits for sensitive embargo schemas). The runtime enforces the declared minimum; commits with shorter nonces are rejected at admission.

Why not bound higher. Longer nonces increase commit-transaction size linearly. 128 bits is the smallest size delivering 128-bit security in the worst case (binary payload space). Larger nonces are governance-tunable per-schema, not protocol-mandated.

5.4 Time-Lock Security

Claim. Reveal-before-`reveal_at` is rejected by validators reading chain height. No exotic cryptography (verifiable delay functions, time-lock encryption) is required.

Argument. The §4.2 reveal admission check verifies $H_{\text{current}} \geq \text{reveal\_at}$. If $H_{\text{current}} < \text{reveal\_at}$, the reveal is rejected at mempool admission. Chain height is the canonical clock; validators reading chain state agree on it by consensus.

In practice. A committer who wants to delay a reveal by ~1 hour sets `reveal_at = H_now + 300` (at 12-second blocks, ~3600 seconds). The reveal cannot land before block 300; honest validators

reject earlier submissions. An adversarial validator who admits an early reveal produces a block other honest validators reject.

Reveal-after-TTL. Symmetric. A reveal submitted at $H_{\text{current}} \geq \text{reveal\_at} + \text{ttl}$ is rejected; the commitment is in EXPIRED state, the runtime knows this from chain state.

Soft real-time only. "1 hour from now" is not exactly an hour; it's 300 blocks at the chain's nominal block time. If block time slows due to network conditions, the real-world time-lock duration grows. This is the intended semantics: chain height is the clock, not wall-clock time. Schemas requiring wall-clock-bounded windows must accept the approximation.

5.5 Hash Function Choice

The schema declares its hash function at registration (§3.4). Three currently-supported choices:

Hash	Output	Collision resistance	Pre-image	Notes
SHA-256	256 bits	2^{128}	2^{256}	Default; universally supported
BLAKE3	256 bits	2^{128}	2^{256}	~5x faster on modern CPUs
Poseidon	256 bits	$\sim 2^{128}$	$\sim 2^{256}$	ZK-friendly; slower verification

All three meet the protocol's collision-and-pre-image-resistance requirements at the 128-bit security level. BLAKE3 is the speed recommendation for high-throughput auction schemas; Poseidon is the only option admitting efficient zero-knowledge reveal variants (§9.1).

Hash is per-schema, not per-commitment. A commitment's hash is implicit from its schema's declaration. There is no per-commitment hash field; this prevents an adversary from submitting a commitment under SHA-256 and trying to reveal under a weaker hash.

Migration. When (or if) any hash is broken cryptanalytically, schemas migrate by registering a new schema-id with a new hash. The §3.4 schema-versioning model (matching native-delegation §10.4 and per-schema-fees §9.5) supports backward-compatible cohorts. Pre-break commitments live out their TTL under the old hash; post-break commitments under the new schema. No retroactive invalidation.

---

6. Use Cases and the Validation Gate

6.1 The Use-Case-Validation Gate

The engineering work for time-locked attestations begins only when at least one design partner per use-case category (auction, embargo, regulatory time-lock) has submitted a concrete use-case description. Until that gate is satisfied, this paper is specification-only.

Three categories. Each unlocks a different surface of the mechanism:

- **Sealed-bid auction**: high-throughput, large-deposit, short-window. Drives the §4.4 deposit-on-commit and §4.5 front-running defense surfaces.
- **Embargoed announcement**: low-throughput, optional-deposit, medium-window. Drives the §3.2 hiding-property surface (the payload must remain hidden until reveal) and the §8.1 never-reveal failure-mode analysis.
- **Regulatory time-lock**: low-throughput, no-deposit, long-window. Drives the long-TTL handling (up to `ttl_max` ~ 14 days) and the §8.2 late-reveal handling for partial-disclosure regimes.

A single use case per category satisfies the gate; subsequent partners can add their own cases within categories. The gate is a quality threshold, not a quota; it ensures the engineering investment is justified by demand across enough of the design space to validate the abstractions.

6.2 Use Case Template

Each design partner submits a description matching this template:

Field 1: Operator and workload. Who runs the workload? What's the expected per-day commit volume? What's the expected reveal-window distribution (mean, p99)?

Field 2: Payload structure. What does the committed payload look like? What's the payload entropy (binary, small-discrete, structured, free-form)? This drives the nonce-length requirement (§3.4).

Field 3: Deposit policy. Is there a per-commit deposit? At what level? Where does it go on cleanup (committer / treasury / burn / cleanup_runner / split)?

Field 4: Slashing requirements. Should never-revealed commitments slash the committer? At what magnitude? §8.1 names this as schema-author choice; the partner specifies what they need.

Field 5: Failure-mode requirements. Which §8 failure mode matters most? Never-reveal, late-reveal, front-running, hash collision, nonce reuse, reveal-DoS. Force a ranking; the engineering tightens the corresponding defense.

Field 6: Integration commitment. Does the partner agree to integrate against the v1.5 protocol during the early-stage pilot? With which timeline?

6.3 Hypothetical Use Cases (Not Yet Validated)

The following are illustrative, not validated. They are NOT commitments to ship until §6.2 descriptions are in hand.

Hypothetical 1: Sealed-bid NFT auction.

Operator: an NFT marketplace running sealed-bid auctions for limited drops. Volume: ~1000 commits per auction, ~5-second reveal window after close. Payload: bid amount (1-10000 range, 14 bits entropy) plus bidder address (32 bytes). Deposit: 10% of bid value (held to prevent commit-and-disappear). Failure-mode priority: front-running between commit and reveal (an adversary observing reveals in mempool would gain unfair information). Status: hypothetical; no marketplace partner has signed up.

Hypothetical 2: Embargoed press release.

Operator: a news outlet committing stories ahead of an embargo time (e.g., quarterly earnings disclosures, government announcements). Volume: 1-10 commits per day. Payload: structured press release (free-form text, low entropy in headline, high entropy overall). Deposit: zero (the operator's reputation is the stake). Failure-mode priority: hiding (the commit must not leak the headline before reveal). Status: hypothetical.

Hypothetical 3: Regulatory time-locked filing.

Operator: a financial filer submitting a regulatory disclosure that must land at deadline but become public only after a retention period (e.g., 30 days). Volume: ~10 commits per quarter. Payload: structured filing document (typically high entropy). Deposit: zero. Failure-mode priority: late-reveal (if the filer fails to reveal by `reveal_at + ttl`, the filing is considered missed and the filer faces regulatory penalties; this is application-layer, not chain-enforced). Status: hypothetical; pending regulatory-compliance partner conversation.

6.4 What §6 Looks Like at v0.2.x

When design partners submit §6.2 descriptions, §6.3 expands to include validated use cases with partner attribution, integration timelines, and any deviations from the v0.2 mechanism partners require. The §6.1 gate is satisfied; the engineering cycle for v1.5 time-locked attestations begins.

Until then, §6.3 hypotheticals are illustrative, not prescriptive. The paper is design-space documentation; the chain ships v1 without time-locked attestations.

---

7. Comparison with Prior Systems

Time-locked attestations occupy a distinct point in the design space against prior commit-reveal mechanisms.

Axis	Off-chain commit-reveal	On-chain auction smart contracts	Drand timelock / VDF	Embargoed storage (Filecoin / Arweave)	Time-locked attestations (this paper)
Where the time-lock lives	Trusted off-chain operator	Per-application smart contract	Cryptographic primitive (delay function)	Per-network retrieval policy	Chain runtime (height-based)
Trust model	Trusted operator (auctioneer)	Trusted contract code (audit-dependent)	Trustless (cryptographic)	Trusted network (storage providers)	Trustless under BFT (chain consensus)
Reveal mechanism	Operator publishes payload	Committer submits reveal transaction	Cryptographic decryption at time T	Retrieve from storage post-embargo	Committer submits reveal transaction
Cost per commit	Off-chain	Contract execution	Off-chain crypto + on-chain decrypt	Storage fee	Single chain tx (§4.1)
Liveness requirement	Operator honest + alive	Contract + reveal-tx liveness	Drand or VDF time	Storage network liveness	Chain BFT liveness
Hash collision concern	N/A (centralized)	SHA-256 per contract	N/A	N/A	Per-schema choice (§5.5)
Cascade integration	No	No	No	No	Yes (via CSC §3.4)
Production status	Live (manual auctions, off-chain protocols)	Live (Vickrey on Ethereum, multiple variants)	Drand live since 2020; VDFs research-grade	Filecoin live; Arweave embargo live	v1.5 specification (this paper)

The four comparators each solve a different subset. **Off-chain commit-reveal** is the simplest deployment but requires trusting the operator. **On-chain auction smart contracts** eliminate that trust by encoding the protocol in a contract; the cost is per-application implementation and audit burden. **Drand timelock encryption** and **VDFs** provide trustless time-locks via cryptographic primitives; the trade-off is that the time-lock is purely cryptographic, not tied to chain state or attestor sets, so integration with PoUA reputation is application-layer. **Embargoed storage networks** address a related but distinct problem (delayed retrieval); the time-lock is at the storage layer, not at the attestation layer.

Time-locked attestations occupies a different point: the time-lock is a runtime primitive on Ligate Chain, enforced by validators reading chain height (no exotic cryptography), tied directly to the attestation/attestor-set/PoUA-reputation stack the chain already supports. The trade-off is that we inherit the chain's BFT liveness assumptions; under network partitions long enough to delay reveal beyond TTL, commitments expire (which is the desired behavior in most use cases, but bears stating).

The unique value-add is **native cascade with attestations**. None of the four comparators model what happens when a revealed attestation is later slashed: this paper integrates cleanly with cross-schema-composition's §3.4 validity state machine, so a downstream attestation referencing a revealed time-locked attestation participates in the cascade defined there.

---

8. Failure Modes

8.1 Never-Reveal

[**v0.2:** Committer commits and never reveals. Defenses: TTL forces cleanup; deposit-on-commit creates economic incentive to reveal; per-schema slash rule for committers exceeding never-reveal threshold.]

8.2 Late-Reveal

[**v0.2:** Committer reveals after reveal_at but before TTL. Default: accepted. Some use cases (auction with hard close) want strict deadline; specified per schema.]

8.3 Front-Running Between Commit and Reveal

[**v0.2:** Adversary observes reveal in mempool, submits competing commit using the now-known payload. Defense: batched-reveal sequencing per §4.5.]

8.4 Hash Collisions

[**v0.2:** SHA-256 collision resistance is computationally infeasible at 128-bit security. Defense: hash-function choice + nonce entropy bound.]

8.5 Nonce Reuse

[**v0.2:** Same nonce used across two commits with different payloads breaks hiding for an adversary who sees both. Defense: nonce-derivation from a deterministic per-commitment seed; specification in v0.2 §4.1.]

8.6 Reveal-DoS

[**v0.2:** Adversary commits many commitments without intent to reveal, bloating chain state. Defense: TTL + cleanup-runner incentive + minimum deposit.]

---

9. Limitations and Future Work

9.1 ZK-Friendly Variant

[**v0.2:** Out of scope. A SNARK-based reveal where the runtime verifies the commitment match without seeing payload would enable confidential-payload time-locks. Research-grade; deferred.]

9.2 Cross-Chain Time-Locks

[**v0.2:** Out of scope. A commitment on Ligate Chain whose reveal is gated by a foreign chain's block height. Separate paper if needed.]

9.3 VDF-Based Hard Time-Locks

[**v0.2:** Out of scope. Verifiable Delay Functions could provide "no early reveal even if the committer wants to", different security model. Not a v1.5 priority.]

9.4 Multi-Party Commit-Reveal

[**v0.2:** Out of scope. M-of-N reveal patterns (any M of N committers can reveal) need additional cryptography. Tracked as follow-up.]

---

10. Conclusion

[**v0.2:** Recap. Time-locked attestations as a runtime primitive eliminate the trusted auctioneer / coordinator from commit-reveal workflows. The cryptographic argument is standard; the operational argument is about TTL, deposit, and front-running defense. v1.5 protocol territory: ships when at least one design partner per category has validated the demand.]

References

[**v0.2:** Pedersen and Lamport commitment papers, Vickrey auction papers, Drand timelock encryption papers, RANDAO specification, Wesolowski VDF, plus standard PoUA references.]

Appendix A: Simulator Validation Plan

[**v0.2:** What `prototypes/time-locked-attestations-sim/` will contain. Test harness for commit-reveal correctness, never-reveal cleanup, front-running defense, nonce-entropy enforcement. Cross-language test vectors for the canonical commitment encoding.]

Appendix B: Formal Definitions

[**v0.2:** Restated definitions of commitment, reveal, nonce, TTL, validity state machine, in formal notation.]